Evaluating Value-Based Decomposition for Scalable Coordination in Multi-Agent Pursuit Environments

Konstantin Bobenko
Hasso Plattner Institute, University of Potsdam
Germany

Abstract

Sparse rewards and mandatory joint coordination make pursuit-evasion a demanding benchmark for multi-agent reinforcement learning. We apply QMIX [2], a monotonic value function factorization method, to the SISL (Stanford Intelligent Systems Laboratory) Pursuit environment and evaluate its performance across grid sizes of 8×8 and 12×12 under both frozen and active evader conditions. By training with access to the global state while keeping execution fully decentralized, QMIX agents learn cooperative capture behaviors that Independent Q-Learning consistently fails to produce. The results show stable convergence on small grids and highlight the scalability challenges that arise as the state space grows and struggles of the approach in some scenarios.

1 Introduction

Many real-world tasks require a team of agents to act jointly under partial information and without centralized control [1]. Multi-Agent Reinforcement Learning (MARL) has emerged as the dominant framework for such settings, extending single-agent RL to the case where multiple agents must learn cooperative policies from shared experience. Yet scaling deep RL to multi-agent settings introduces two compounding difficulties. First, as each agent's policy changes during learning, the environment appears *non-stationary* from the perspective of every other agent, undermining the convergence guarantees that single-agent RL relies upon. Second, when agents share a common reward signal, the *credit assignment* problem asks which agent's actions actually caused a good outcome—a question that becomes exponentially harder as team size grows. In the Pursuit setting, for instance, all pursuers receive the same capture bonus regardless of whether a given agent occupied an optimal flanking position, made the decisive contact, or was spatially distant from the capture event entirely. The shared reward signal provides no mechanism to distinguish these contributions.

Pursuit-evasion tasks crystallize both of these challenges. In the Pursuit environment [1], a team of pursuers must coordinate their movements to trap one or more evaders on a discrete grid. Capture only occurs once a sufficient number of pursuers surround an evader, meaning that a single agent acting optimally in isolation will never collect a reward. The task therefore demands genuine inter-agent coordination rather than merely parallel execution of independent skills.

Independent Q-Learning (IQL) is the natural baseline: each agent trains its own Q-network while ignoring the non-stationarity introduced by its teammates. While IQL can succeed on loosely coupled tasks, it has no mechanism for representing the joint contribution of multiple agents to a shared outcome. In the Pursuit domain, this causes IQL agents to fail to converge on coordinated encirclement strategies.

This paper presents an empirical evaluation of QMIX on the SISL Pursuit task. QMIX factorizes the joint action-value function into a monotonic combination of per-agent utilities, allowing centralized training with a shared global reward while guaranteeing that each agent can act greedily and independently at execution time. We examine how this Centralized Training, Decentralized Execution-approach scales across varying grid sizes and whether it remains effective when evaders pursue an active escape strategy.

2 Background & Problem Formulation

We model the Pursuit task as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP), defined by the tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, r, \Omega, n, \gamma \rangle,$$

where the components are:

- $\mathcal{S}$: the global state space. A state $s \in \mathcal{S}$ fully describes the grid at a given timestep.
- $\mathcal{A}$: the per-agent action space, shared by all pursuers. Each agent i picks one action $u^i \in \mathcal{A}$, and together they form the joint action $\mathbf{u} = (u^1, \ldots, u^n)$.
- P: the transition function. Given the current state s and joint action $\mathbf{u}$, it defines the probability $P(s' \mid s, \mathbf{u})$ of reaching the next state s'.
- r: the shared team reward. The environment only returns one single group reward $r(s, \mathbf{u})$ after each step—there is no per-agent reward.
- Ω: the per-agent observation space. Each agent sees only a small 7×7 patch of the grid centered on its own position;
- n: the number of pursuer agents.
- $\gamma \in [0, 1)$: the discount factor, controlling how much agents value future rewards.

Because each agent sees only its local window rather than the full state, it must base its decisions on everything it has observed and done so far—its action-observation history. The goal is to find a joint policy that maximizes the team's total discounted reward over an episode.

2.1 The Pursuit Task

Our experiments use discrete grids of size 8×8 and 12×12. Each pursuer observes a 7×7 patch centered on its own position, encoded as a three-channel tensor of shape 7×7×3: the first channel marks obstacle positions, the second marks all other pursuers, and the third marks evaders. This gives each agent a structured local view of its immediate surroundings, flattened to a 147-dimensional input vector before being passed to the network. The agent may take one of five actions: {Up, Down, Left, Right, Stay}. Capture is configurable via two parameters: *n_catch*, which triggers a capture when at least n pursuers are adjacent to an evader, and **surround**,

which triggers a capture when an evader is completely enclosed by pursuers or grid boundaries. We evaluate both frozen evaders (which do not move) and active evaders (which pursue a simple escape heuristic), with the active setting forming a strictly harder variant of the task.

2.2 Reward Structure

The reward signal is sparse and shared. A single team reward is generated by the environment and distributed to all pursuers simultaneously [?]. The team receives +5 upon a successful capture—large enough to be the dominant learning signal once agents discover cooperative behavior—and a small +0.01 tag reward each time a pursuer is adjacent to an evader, which discourages purely passive strategies. A per-step penalty of −0.1 keeps episodes from running indefinitely. No individual credit is assigned—every pursuer receives the same reward signal regardless of its specific contribution to a capture. This design makes the multi-agent credit assignment problem unavoidable and rules out strategies where a single dominant agent can earn rewards without cooperation.

Providing separate rewards to each agent would make the learning problem significantly easier but would not reflect the cooperative nature of the task. If agents were rewarded individually—for example only the agents directly involved in a capture—they could learn selfish strategies that maximize their own reward rather than coordinating with the team. In extreme cases, agents might even compete for captures or avoid assisting others if doing so does not increase their personal reward. Using a single shared team reward instead forces agents to learn cooperative behavior.

3 Methodology

3.1 Agent Architecture

Each pursuer is controlled by a three-layer feed-forward Deep Q-Network (DQN) that takes the flattened 7×7×3 = 147-dimensional observation as input and outputs a Q-value for each of the five actions. All agents share a single set of network weights (parameter sharing). Since all pursuers are homogeneous—same action space, same observation structure, same role—there is no reason to give each agent its own network. It also means that every experience, regardless of which pursuer produced it, contributes to the same gradient update, which substantially improves sample efficiency. Agents still behave differently in practice because they occupy different positions and therefore receive different observations—two agents in different situations will produce different Q-values from the very same network.

Initially, each agent was given its own independent network. However, this turned out to be unnecessary and counterproductive: coordination was very hard to learn, training was highly unstable, and reward grew far more slowly. The individual networks added complexity without any benefit, since the agents are fully symmetric and do not require specialized policies. Switching to a single shared network stabilized training and improved the reward signal, making it the more appropriate choice for this setting.

3.2 The Mixing Network

The core of QMIX is a mixing network that aggregates the per-agent utilities Q^a into a global Q_{tot}. To ensure that greedy action selection is consistent across the individual and joint levels—so that $\text{argmax}_{\mathbf{u}}\, Q_{\text{tot}}$ decomposes into N independent argmax operations—the mixer must satisfy the monotonicity constraint:

$$\frac{\partial Q_{\text{tot}}(s, \mathbf{u})}{\partial Q^a(\tau^a, u^a)} \geq 0 \quad \forall\, a \in \{1, \ldots, N\}. \tag{1}$$

This is enforced architecturally: the weights of the mixing network are produced by the **Mixer Network** that takes the global state s as input. Passing these weights through an absolute value function guarantees non-negativity throughout. The biases are produced by standard linear layers conditioned on s, allowing the mixer to shift the combined value without violating monotonicity.

3.3 Centralized Training, Decentralized Execution

During training, the hypernetworks have access to the full global state s—the positions of all agents and evaders. The joint loss is the mean-squared Bellman error:

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(Y^{\text{tot}} - Q_{\text{tot}}(s, \mathbf{u};\, \theta)\big)^2\Big]. \tag{2}$$

The target $Y^{\text{tot}} = r + \gamma\, Q_{\text{tot}}^{\text{target}}(s', \mathbf{u}';\, \theta^-)$ is computed from a periodically updated target network, with $\mathbf{u}'$ chosen greedily by the online networks (Double DQN).

At execution time, the mixing network and global state are discarded entirely. Each agent selects its action by $\text{argmax}_{u^a} Q^a(\tau^a, u^a)$ using only its local observation. Coordination emerges not from communication but from the structure of the learned individual utilities, which were shaped during training to be globally consistent. Table 1 summarizes which components are active during each phase.

Table 1: Components active during centralized training versus decentralized execution.

Component	Training	Execution
Global state	✓	×
Mixer + hypernetworks	✓	×
Agent network	✓	✓
Communication	×	×

3.4 Training Stability

Beyond the target network, we incorporate two further standard techniques. An experience replay buffer stores environment transitions and provides decorrelated minibatches for gradient updates. Epsilon-greedy exploration starts at $\epsilon = 1.0$ and decays linearly to 0.05 over 50% of total training steps, ensuring the agents explore the joint state space broadly before committing to a policy.

3.5 Evaluation Protocol

Training runs for 1000 episodes in total. Every 50 episodes, training is paused and the current policy is evaluated over 10 dedicated evaluation episodes. During evaluation, exploration is reduced to $\epsilon = 0.05$ and no gradient updates are performed. The average reward across these 10 episodes is recorded as a single data point.

This produces 20 evaluation points per run (plus one before training starts at episode 0), which are the values plotted in all graphs throughout this paper.

3.6 The Global State of the Mixer Network

The mixing network requires a global state s to condition its hypernetwork weights. Rather than using the raw grid observation, we construct a compact state vector from the ground-truth positions of all agents. Concretely, s is a fixed-length vector of normalized (x, y) coordinates: the first $2n_{\text{pursuers}}$ entries hold the pursuer positions, followed by $2n_{\text{evaders}}$ entries for the evaders, giving a total dimension of $2n_{\text{pursuers}} + 2n_{\text{evaders}}$. All coordinates are normalized to $[0, 1]$ by dividing by the respective grid dimensions.

Importantly, the state vector does not shrink as evaders are captured. Each evader occupies a fixed slot in the vector for the entire episode; once caught, its slot is filled with $(0.0, 0.0)$ rather than being removed. This keeps the state dimension constant throughout an episode, which is required by the hypernetworks since their weight matrices must have a fixed input size. The zero-padding also carries an implicit signal to the mixer: a slot at the origin indicates a captured evader, allowing the network to infer how many targets remain without any additional bookkeeping.

4 Experimental Results

4.1 Task Comparison per Grid Size

Figures 1 and 2 compare all four task variants within each grid size. On the 8×8 grid, agents face a denser environment and shorter inter-agent distances, while the 12×12 grid spreads agents and evaders across a larger state space.

This effect is not noticable if comparing the learning rates and final results of the Experiment types across the different Enviourments. The Learning curve improvement are very similar across the different experiment types and the final rewards are very similar.

Table 2: Best average evaluation reward achieved per strategy and grid size.

Strategy	8×8	12×12
Surround Freeze	−27.68	−38.17
Surround Active	14.18	15.79
Touch Freeze	64.13	69.43
Touch Active	46.22	44.73

Table 2 reveals a clear hierarchy across strategies that is consistent across both grid sizes. The Touch variants outperform the Surround variants by a wide margin. Touch Freeze achieves peak rewards of +64 and +69 on the 8×8 and 12×12 grids respectively, while Touch Active reaches +46 and +45. The Surround variants lag far behind, with Surround Active peaking near +14–+16 and Surround Freeze failing to learn altogether.

The strong performance of the Touch variants can be attributed to the relative simplicity of the required coordination pattern. Under the Touch capture condition, only multiple pursuer need to make contact with an evader at the same time, which means those agents can develop a straightforward division of labor. Noticing a pattern of touching the same square at the same time seems to be easier then learning to surround the target.

Surround Freeze, by contrast, fails to show any learning signal across both grid sizes. The Surround capture condition requires multiple pursuers to simultaneously occupy adjacent cells around an evader, demanding tightly synchronized multi-agent behavior. In isolation this is already a harder credit assignment problem, but the freeze variant compounds the difficulty in a subtle way: because evaders are stationary, the environment is effectively static. Agents receive the same local observations repeatedly whenever they are near an unoccupied evader. This creates ideal conditions for agents to fall into infinite loops, repeatedly cycling through the same sequence of moves without ever triggering a capture. With no dynamic pressure forcing them to adapt and a sparse reward signal that only fires on successful surrounding, the agents never receive the feedback necessary to break out of these degenerate cycles, leaving the policy stuck near its random initialization for the entire training run.

In the following sections, we will analyze the emergent behaviors across all task variants to evaluate their respective performance and identify the underlying causes for these strategic discrepancies.

4.2 Scaling: Surround / Freeze

The Surround/Freeze condition provides the clearest illustration of the infinite-loop failure mode described above, and the scaling comparison confirms that neither grid size escapes it. The 8×8 reward curve remains flat near −28 across all 1000 training episodes, while the 12×12 curve is even more negative, hovering around −38 throughout. The larger grid slightly worsens the situation: the increased distance between agents and evaders makes accidental co-location of two pursuers on adjacent cells even rarer, so the agents receive capture events even less frequently and have even less signal to work with. Neither run shows any upward trend at any point during training. This condition is the one case in our experiments where QMIX completely fails to make progress, and the cause is the combination of a demanding joint capture condition and a static target that removes the only pressure that could otherwise force the agents out of degenerate action cycles.

4.3 Scaling: Surround / Active

The critical difference introduced by an active evader is that the environment is no longer static. In the Freeze condition, agents could revisit the exact same local observation indefinitely, which is what allowed degenerate action cycles to persist. Here, by the time agents return to a similar spatial configuration, the evader has already moved the state space has shifted, and the same sequence of actions no longer produces the same outcome. This continuous perturbation prevents the infinite-loop failure mode entirely, and both grid sizes benefit equally from it. The results confirm this: the 8×8 grid peaks at +14 and the 12×12 grid at +16, an almost negligible difference. Scaling to the larger map therefore costs nothing in this condition, because the active evader's movement already provides the kind of environmental variety that forces ongoing adaptation regardless of map size. The remaining performance ceiling is set by the difficulty of the Surround capture condition itself, not by the grid.

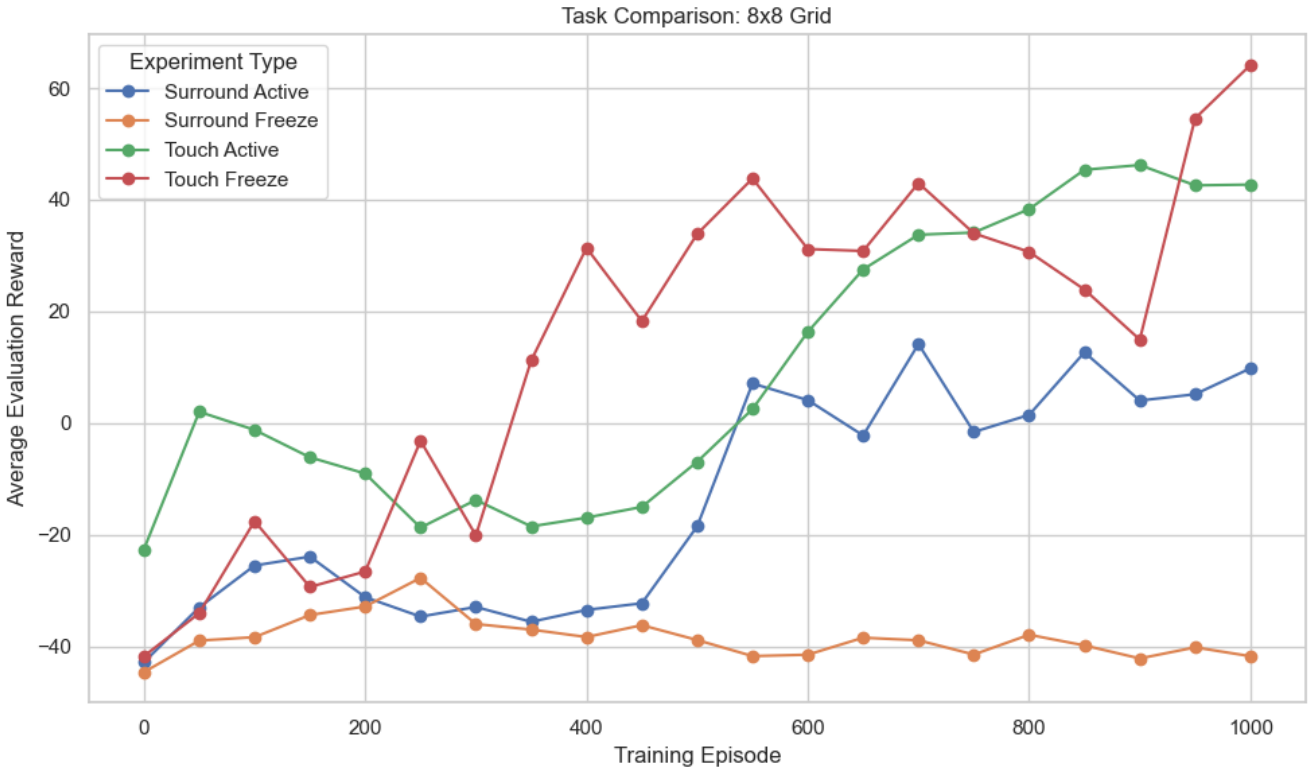

Figure 1: Average evaluation reward on the 8×8 grid for all four task types across training.

4.4 Scaling: Touch / Freeze

The Touch/Freeze condition relaxes the capture requirement relative to the Surround variants, and this manifests clearly in the scaling comparison. On both grid sizes, convergence is faster and the asymptotic reward is higher than the corresponding Surround/Freeze runs. The reduced coordination overhead—fewer agents need to achieve precise joint positioning—makes the credit assignment problem more tractable and allows the mixing network to identify productive joint actions more efficiently. This comparison highlights that the difficulty of the MARL credit assignment problem is as sensitive to the capture condition as it is to grid size, and that relaxing task constraints can substantially extend the range of environments in which QMIX converges reliably.

4.5 Scaling: Touch / Active

The Touch/Active condition combines the two properties that individually resolved the problems seen in the other variants: the Touch capture condition makes coordination easy, and the active evader prevents infinite loops. The agents therefore have no major obstacle to overcome, and both grid sizes reflect this peaking at +46 on the 8×8 grid and +45 on the 12×12 grid. The active evader again ensures that the state space keeps changing, so the same flanking strategy that works on the smaller grid transfers directly to the larger one without any degradation in final performance.

5 Discussion

Scalability. Going into this experiment, the expectation was that scaling from 8×8 to 12×12 would clearly hurt performance—more ground to cover, sparser encounters, harder exploration. That mostly did not happen. In three of the four conditions the peak rewards are nearly identical across grid sizes, and Touch/Freeze actually does slightly better on the larger grid (+69 vs. +64). What does get worse is training stability: curves are noisier and convergence takes longer, roughly 50–100 episodes. But the final level the agents reach is not really affected.

The exception is Surround/Freeze, where both grid sizes fail completely. But that is less a scalability problem and more a fundamental issue with the condition itself.

Capture Condition as the Dominant Factor. If there is one clear takeaway from the results, it is that the capture condition matters far more than grid size. Switching from Surround to Touch is worth 30–50 reward points; scaling up the map costs almost nothing. The reason Surround/Freeze fails so badly is that the combination of a hard joint capture requirement and a completely static environment gives agents no pressure to change behavior—they fall into loops and stay there. The active evader fixes this not by making the task easier, but by keeping the environment moving enough that the same action sequence stops producing the same result.

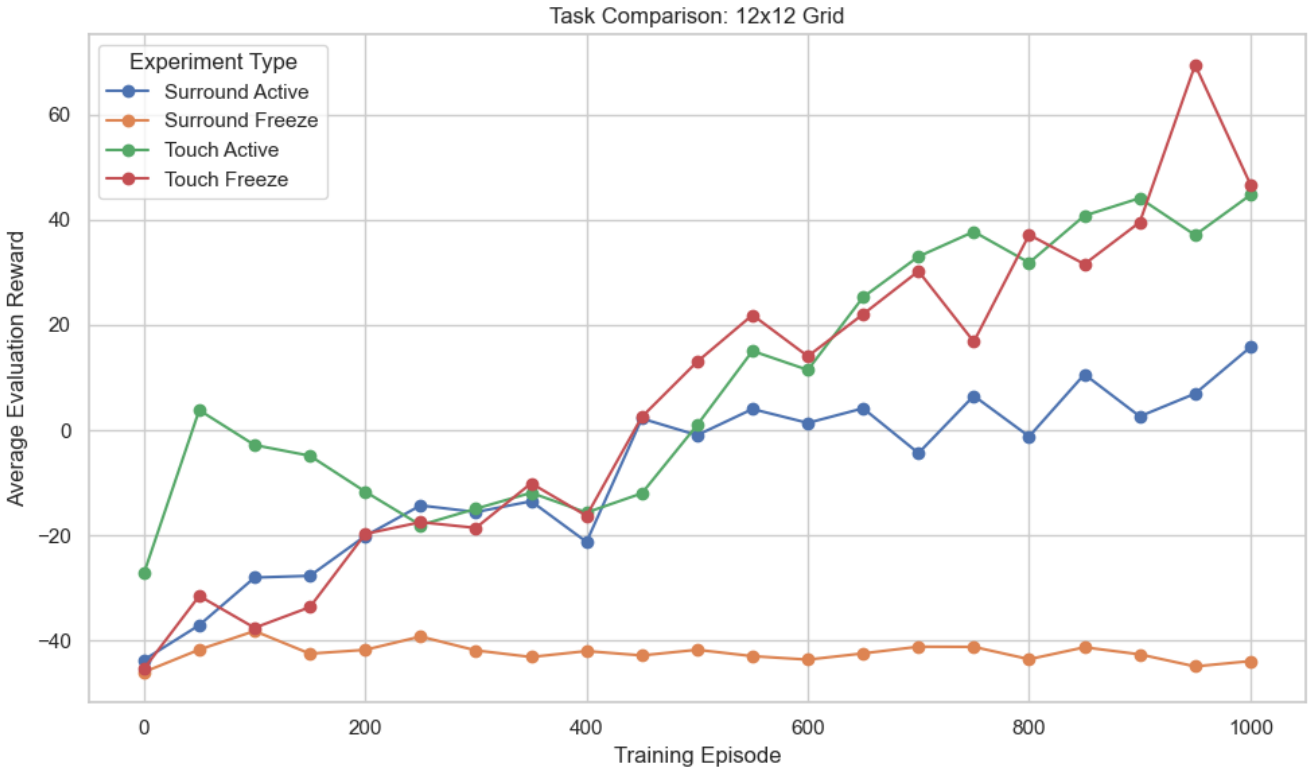

Figure 2: Average evaluation reward on the 12×12 grid for all four task types across training.

Partial Observability and Memory. One limitation that was not fully appreciated during implementation is the lack of agent memory. The feed-forward network sees only the current timestep, so if an evader walks out of the 7×7 window the agent effectively forgets it exists. For the active-evader conditions this probably hurts more than the numbers show. Adding a GRU to the agent network would be a natural next step and likely the single change with the most impact on those conditions.

Critical Reflection on the Implementation. A few choices in hindsight could have been done better. The global state zero-pads caught evaders with $(0.0, 0.0)$ to keep the vector size fixed, but this is technically ambiguous—a live evader at the true grid origin normalizes to the same value. It probably does not matter in practice since that corner is rare, but a binary mask alongside the coordinates would be cleaner.

Evaluation was run at $\epsilon = 0.05$ rather than fully greedy to avoid deterministic loops, but this adds noise to every reported number. Training also ran for a fixed 1000 episodes regardless of whether a condition had already converged or was still improving, which makes the comparison across conditions somewhat uneven. And no ablation was run to check whether the mixing network's state-conditioned weights actually help over plain summation—it would be useful to know how much of the gain is from QMIX specifically versus just from parameter sharing.

Future Work. The Surround/Freeze failure is the most tractable open problem. Curriculum learning—starting agents in the active-evader condition to eliminate early action loops, then transitioning to frozen evaders once a baseline capture behavior is established—seems the most principled intervention. Prioritized experience replay [3] could complement this by increasing the sampling frequency of the rare successful capture transitions, accelerating credit assignment in conditions where reward events are sparse.

The lack of temporal memory compounds this. The feed-forward agent network has no recurrent state, so when an evader exits the 7×7 observation window, the agent retains no information about that evader's last known position or movement direction. A Gated Recurrent Unit (GRU) layer in the agent network would allow agents to maintain a belief across timesteps, and is likely the single architectural change with the greatest potential impact on active-evader performance.

A longer-term question is whether explicit inter-agent communication can address the Surround coordination bottleneck. All coordination in the current system is implicit, encoded into individual Q-values during joint training. The characteristic failure mode in Surround conditions—multiple pursuers converging from the same direction without knowledge of each other's trajectories—is precisely the problem that targeted message passing could resolve. Broadcasting intended approach directions or target assignments

could enable stable role differentiation, a behavioral property that the current architecture cannot express.

6 Conclusion

This paper evaluated QMIX on the SISL Pursuit task across two grid sizes and four task variants. The main finding is that the capture condition, not the grid size, is what determines whether agents learn anything useful. Touch variants converge reliably on both grids; Surround/Active learns something but plateaus low; Surround/Freeze fails entirely because agents get stuck in action loops that the static environment never breaks. Grid size mostly affects training noise and convergence speed rather than the final reward level. The results suggest that QMIX with parameter sharing is a reasonable baseline for cooperative pursuit, but that task design—particularly avoiding static environments with hard joint capture conditions—matters as much as the algorithm itself.

References

[1] Jayesh K. Gupta, Maxim Egorov, and Mykel Kochenderfer. 2017. Cooperative Multi-Agent Control Using Deep Reinforcement Learning. In *Autonomous Agents and Multi-Agent Systems (AAMAS)*. 66–83.

[2] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. 2018. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 4295–4304.

[3] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. 2016. Prioritized Experience Replay. In *International Conference on Learning Representations (ICLR)*.